

```

/*
=====
=====

    STUDENT SCHOLARSHIP MANAGEMENT - SQL
    LAB (SQL Server / T-SQL)
    Run section by section in SSMS. Batches
    separated by GO.

=====
===== */

CREATE DATABASE ScholarshipDB;
GO
USE ScholarshipDB;
GO

/*
=====
=====

    1a. DDL, DML, DCL, TCL

=====
===== */

-- ----- DDL : CREATE -----
CREATE TABLE Student (
    StudentID      INT           PRIMARY
KEY,
    FirstName      VARCHAR(30)   NOT NULL,
    LastName       VARCHAR(30)   NOT NULL,

```

```

        Email          VARCHAR(60)    UNIQUE,
        DOB            DATE,
        Gender         CHAR(1)        CHECK
        (Gender IN ('M','F','O')),
        Department     VARCHAR(20),
        CGPA           DECIMAL(4,2)   CHECK
        (CGPA BETWEEN 0 AND 10),
        AnnualIncome   DECIMAL(12,2)  CHECK
        (AnnualIncome >= 0)
    );

CREATE TABLE Scholarship (
    ScholarshipID INT          PRIMARY
    KEY,
    SchName        VARCHAR(50)  NOT NULL
    UNIQUE,
    Provider       VARCHAR(40),
    Amount         DECIMAL(10,2) CHECK
    (Amount > 0),
    MinCGPA        DECIMAL(4,2),
    MaxIncome      DECIMAL(12,2)
);

CREATE TABLE Application (
    AppID          INT          PRIMARY
    KEY,
    StudentID      INT          NOT NULL,
    ScholarshipID  INT          NOT NULL,
    AppDate        DATE         DEFAULT
    GETDATE(),
    Status         VARCHAR(10)  DEFAULT
    'Pending'
    CHECK (Status IN
    ('Pending','Approved','Rejected')),

```

```

        CONSTRAINT FK_App_Student FOREIGN KEY
        (StudentID)
            REFERENCES Student(StudentID),
        CONSTRAINT FK_App_Sch FOREIGN KEY
        (ScholarshipID)
            REFERENCES
        Scholarship(ScholarshipID)
    );

CREATE TABLE Disbursement (
    DisbID      INT IDENTITY(1,1) PRIMARY
KEY,
    AppID       INT NOT NULL,
    Amount      DECIMAL(10,2) CHECK (Amount >
0),
    DisbDate    DATE,
    Mode        VARCHAR(20),
    CONSTRAINT FK_Disb_App FOREIGN KEY
(AppID) REFERENCES Application(AppID)
);
GO

-- ----- DDL : ALTER -----
ALTER TABLE Student ADD Phone
VARCHAR(15);          -- add column
ALTER TABLE Student ALTER COLUMN Phone
VARCHAR(20);         -- modify column
EXEC sp_rename 'Student.Phone', 'Mobile',
'COLUMN';            -- rename column
ALTER TABLE Student DROP COLUMN
Mobile;               -- drop column
GO

-- ----- DDL : scratch table for

```

```

TRUNCATE / DROP -----
CREATE TABLE TempLog (LogID INT, Msg
VARCHAR(50));
INSERT INTO TempLog VALUES (1,'a'),(2,'b');
TRUNCATE TABLE TempLog;      -- removes all
rows, keeps structure
DROP TABLE TempLog;          -- removes
table itself
GO

-- ----- DML : INSERT -----
INSERT INTO Student VALUES
(1,'Aarav','Sharma','aarav@uni.edu','2003-
05-14','M','CSE', 9.10,180000),
(2,'Diya','Nair','diya@uni.edu','2002-11-
02','F','ECE', 8.50,250000),
(3,'Rohan','Gowda','rohan@uni.edu','2003-
01-21','M','CSE', 7.80,120000),
(4,'Meera','Iyer','meera@uni.edu','2002-08-
30','F','MECH',8.90, 90000),
(5,'Kiran','Patel','kiran@uni.edu','2003-
03-09','M','ECE', 6.90,300000),
(6,'Sneha','Reddy','sneha@uni.edu','2002-
12-17','F','CSE', 9.40,150000),
(7,'Arjun','Das','arjun@uni.edu','2003-07-
25','M','MECH',7.20,200000),
(8,'Pooja','Rao','pooja@uni.edu','2003-09-
12','F','ECE', 8.10,110000);

INSERT INTO Scholarship VALUES
(101,'Merit Excellence Award','State
Govt',50000,9.00,500000),
(102,'Need Based Grant','Central
Govt',40000,6.50,150000),

```

```
(103,'Women in Tech Scholarship','Corporate
Trust',60000,8.00,300000),
(104,'Sports Achiever Award','State
Govt',30000,6.00,400000);
```

```
INSERT INTO Application
(AppID,StudentID,ScholarshipID,AppDate,Stat
us) VALUES
(1,1,101,'2026-06-10','Approved'),
(2,6,101,'2026-06-11','Approved'),
(3,3,102,'2026-06-12','Pending'),
(4,4,102,'2026-06-13','Approved'),
(5,2,103,'2026-06-14','Pending'),
(6,6,103,'2026-06-15','Rejected'),
(7,5,104,'2026-06-16','Rejected'),
(8,7,102,'2026-06-17','Pending');
```

```
INSERT INTO Disbursement
(AppID,Amount,DisbDate,Mode) VALUES
(1,50000,'2026-07-01','Bank Transfer'),
(2,50000,'2026-07-01','Bank Transfer'),
(4,40000,'2026-07-05','Cheque');
GO
```

```
-- ----- DML : UPDATE / DELETE /
SELECT -----
UPDATE Student      SET CGPA = 7.10 WHERE
StudentID = 5;
UPDATE Scholarship SET Amount = Amount *
1.10 WHERE Provider = 'State Govt';
```

```
INSERT INTO Student VALUES
(9,'Temp','User','temp@uni.edu','2004-01-
01','O','CSE',5.00,50000);
```

```

DELETE FROM Student WHERE StudentID = 9;

SELECT * FROM Student;
SELECT FirstName, CGPA FROM Student WHERE
CGPA > 8 ORDER BY CGPA DESC;
GO

/*
=====
=====
1b. DCL and TCL

=====
===== */

-- ----- DCL : GRANT / DENY / REVOKE -
-----
CREATE LOGIN scholar_login WITH PASSWORD =
'Str0ng@Pass123';
CREATE USER scholar_user FOR LOGIN
scholar_login;

GRANT SELECT, INSERT, UPDATE ON Student TO
scholar_user;
DENY DELETE ON Student TO scholar_user;
REVOKE UPDATE ON Student FROM scholar_user;

-- Verify permissions
SELECT dp.name AS UserName,
p.permission_name, p.state_desc,
OBJECT_NAME(p.major_id) AS ObjName
FROM sys.database_permissions p
JOIN sys.database_principals dp ON
p.grantee_principal_id = dp.principal_id

```

```

WHERE dp.name = 'scholar_user';
GO

-- ----- TCL : COMMIT / ROLLBACK /
SAVEPOINT -----
BEGIN TRANSACTION;
    UPDATE Student SET CGPA = 9.90 WHERE
StudentID = 1;
    SAVE TRANSACTION
sp1;                -- savepoint
    UPDATE Student SET CGPA = 1.00 WHERE
StudentID = 2;
    ROLLBACK TRANSACTION
sp1;                -- undo only the 2nd
update
COMMIT
TRANSACTION;                --
1st update is saved
SELECT StudentID, CGPA FROM Student WHERE
StudentID IN (1,2);

BEGIN TRANSACTION;
    DELETE FROM Disbursement;
ROLLBACK
TRANSACTION;                --
everything undone
SELECT COUNT(*) AS DisbRows FROM
Disbursement;
GO

/*
=====
=====
1b (SQL Operations) c) INTEGRITY

```

CONSTRAINTS

Table-level constraints were defined in CREATE; here we add via ALTER and test violations.

```
=====
===== */
```

```
-- Add constraints after creation
ALTER TABLE Scholarship ADD CONSTRAINT
CK_MinCGPA CHECK (MinCGPA BETWEEN 0 AND
10);
ALTER TABLE Student      ADD CONSTRAINT
UQ_Student_Name_DOB UNIQUE (FirstName,
LastName, DOB);
GO
```

-- Test violations (each is caught so the script keeps running)

```
BEGIN TRY -- PRIMARY KEY violation
(duplicate id)
```

```
    INSERT INTO Student VALUES
(1,'X','Y','x@uni.edu','2003-01-
01','M','CSE',7,1000);
```

```
END TRY
```

```
BEGIN CATCH PRINT 'PK error: ' +
ERROR_MESSAGE(); END CATCH;
```

```
BEGIN TRY -- UNIQUE violation (duplicate
email)
```

```
    INSERT INTO Student VALUES
(20,'X','Y','aarav@uni.edu','2003-01-
01','M','CSE',7,1000);
```

```
END TRY
```

```

BEGIN CATCH PRINT 'UNIQUE error: ' +
ERROR_MESSAGE(); END CATCH;

BEGIN TRY  -- CHECK violation (CGPA > 10)
    INSERT INTO Student VALUES
(21,'X','Y','y@uni.edu','2003-01-
01','M','CSE',11,1000);
END TRY
BEGIN CATCH PRINT 'CHECK error: ' +
ERROR_MESSAGE(); END CATCH;

BEGIN TRY  -- FOREIGN KEY violation
(student 99 does not exist)
    INSERT INTO Application
(AppID,StudentID,ScholarshipID) VALUES
(50,99,101);
END TRY
BEGIN CATCH PRINT 'FK error: ' +
ERROR_MESSAGE(); END CATCH;

BEGIN TRY  -- FK on delete (student 1 is
referenced)
    DELETE FROM Student WHERE StudentID =
1;
END TRY
BEGIN CATCH PRINT 'FK delete error: ' +
ERROR_MESSAGE(); END CATCH;
GO

-- Drop a constraint
ALTER TABLE Scholarship DROP CONSTRAINT
CK_MinCGPA;
GO

```

```

/*
=====
=====
1b d) BUILT-IN FUNCTIONS
=====
===== */

-- ----- Date -----
SELECT
GETDATE()
AS Today,
    DATEADD(DAY, 30,
GETDATE()) AS After30Days,
    DATEDIFF(YEAR, '2003-05-14',
GETDATE()) AS AgeYears,
    DATEPART(MONTH,
GETDATE()) AS MonthNo,
    DATENAME(WEEKDAY,
GETDATE()) AS DayName,

EOMONTH(GETDATE())
AS MonthEnd,
    YEAR(GETDATE()) AS Yr,
MONTH(GETDATE()) AS Mth, DAY(GETDATE()) AS
Dy;

SELECT FirstName, DOB, DATEDIFF(YEAR, DOB,
GETDATE()) AS Age FROM Student;
SELECT AppID, AppDate, DATEDIFF(DAY,
AppDate, GETDATE()) AS DaysSinceApplied
FROM Application;

-- ----- Time -----

```

```

SELECT
SYSDATETIME()
AS HighPrecisionNow,
    CAST(GETDATE() AS
TIME)          AS CurrentTime,
    CONVERT(VARCHAR(8), GETDATE(),
108)          AS HHMMSS,
    DATEPART(HOUR,
GETDATE())    AS Hr,
    DATEPART(MINUTE,
GETDATE())    AS Mi,
    DATEPART(SECOND,
GETDATE())    AS Sec,
    DATEADD(HOUR, 5,
GETDATE())    AS Plus5Hours;

-- ----- Numeric -----
SELECT ABS(-45.5)      AS AbsVal,
    CEILING(7.2)      AS Ceil,
    FLOOR(7.8)        AS Flr,
    ROUND(8.567, 1)   AS Rnd,
    POWER(2, 5)       AS Pow,
    SQRT(144)         AS Sqrt,
    SIGN(-9)          AS Sgn,
    10 % 3            AS Modulo,
    PI()              AS PiVal;

SELECT SchName, Amount, ROUND(Amount *
0.18, 2) AS GST_18pct FROM Scholarship;
SELECT SUM(Amount) AS Total, AVG(Amount) AS
Average, MAX(Amount) AS Highest,
    MIN(Amount) AS Lowest, COUNT(*) AS
Cnt FROM Scholarship;

```

```

-- ----- String -----
SELECT
UPPER(FirstName)                                AS
Upper,

LOWER(LastName)                                AS
Lower,

LEN(FirstName)                                AS
Len,
        LEFT(FirstName,
3)                                AS Left3,
        RIGHT(FirstName,
2)                                AS Right2,
        SUBSTRING(Email, 1, CHARINDEX('@',
Email) - 1) AS UserPart,
        CONCAT(FirstName, ' ',
LastName) AS FullName,
        REPLACE(Email, 'uni.edu',
'college.edu') AS NewEmail,

REVERSE(FirstName)                                AS
Rev,
        LTRIM(RTRIM('  trim me
')) AS Trimmed,
        STUFF(FirstName, 1, 1,
'#') AS Stuffed
FROM Student;

-- ----- Conversion -----
SELECT CAST(9.75 AS
INT)                                AS
CastToInt,
        CAST(GETDATE() AS

```

```

DATE) AS CastToDate,
        CONVERT(VARCHAR(10), GETDATE(),
103) AS DDMMYYYY,
        CONVERT(VARCHAR(10), GETDATE(),
101) AS MMDDYYYY,
        TRY_CAST('abc' AS
INT) AS
TryCastFail_NULL,
        TRY_CONVERT(INT,
'123') AS
TryConvertOK,
        PARSE('15-08-2026' AS DATE USING
'en-GB') AS ParsedDate,
        'CGPA is ' + CAST(9.1 AS
VARCHAR(5)) AS Concatenated;
GO

```

```

/*

```

```

=====
=====

```

```

1c e) JOINS

```

```

=====
===== */

```

```

-- INNER JOIN : students with the
scholarships they applied for
SELECT s.FirstName, sc.SchName, a.Status
FROM Student s
INNER JOIN Application a ON s.StudentID =
a.StudentID
INNER JOIN Scholarship sc ON
a.ScholarshipID = sc.ScholarshipID;

```

```
-- LEFT JOIN : all students, even those who
never applied (Pooja)
SELECT s.FirstName, a.AppID, a.Status
FROM Student s
LEFT JOIN Application a ON s.StudentID =
a.StudentID;
```

```
-- Students who never applied
SELECT s.FirstName
FROM Student s
LEFT JOIN Application a ON s.StudentID =
a.StudentID
WHERE a.AppID IS NULL;
```

```
-- RIGHT JOIN : all applications, with
disbursement details if any
SELECT a.AppID, a.Status, d.Amount,
d.DisbDate
FROM Disbursement d
RIGHT JOIN Application a ON d.AppID =
a.AppID;
```

```
-- FULL OUTER JOIN : students and
applications, unmatched from both sides
SELECT s.FirstName, a.AppID
FROM Student s
FULL OUTER JOIN Application a ON
s.StudentID = a.StudentID;
```

```
-- CROSS JOIN : every student x every
scholarship
SELECT s.FirstName, sc.SchName
FROM Student s CROSS JOIN Scholarship sc;
```

```

-- SELF JOIN : pairs of students in the
same department
SELECT s1.FirstName AS Student1,
s2.FirstName AS Student2, s1.Department
FROM Student s1
JOIN Student s2 ON s1.Department =
s2.Department AND s1.StudentID <
s2.StudentID;

-- Multi-table with aggregate : total money
disbursed per student
SELECT s.FirstName, SUM(d.Amount) AS
TotalReceived
FROM Student s
JOIN Application a ON s.StudentID =
a.StudentID
JOIN Disbursement d ON a.AppID = d.AppID
GROUP BY s.FirstName;
GO

/*
=====
=====
1c f) SUB-QUERIES AND CORRELATED SUB-
QUERIES

=====
===== */

-- Scalar sub-query : students above
overall average CGPA
SELECT FirstName, CGPA FROM Student
WHERE CGPA > (SELECT AVG(CGPA) FROM
Student);

```

```

-- IN : students who applied for any State
Govt scholarship
SELECT FirstName FROM Student
WHERE StudentID IN (
    SELECT a.StudentID FROM Application a
    JOIN Scholarship sc ON a.ScholarshipID
    = sc.ScholarshipID
    WHERE sc.Provider = 'State Govt');

-- NOT IN : scholarships nobody has been
approved for
SELECT SchName FROM Scholarship
WHERE ScholarshipID NOT IN (SELECT
ScholarshipID FROM Application WHERE Status
= 'Approved');

-- ALL / ANY
SELECT SchName, Amount FROM Scholarship
WHERE Amount > ALL (SELECT Amount FROM
Scholarship WHERE Provider = 'Central
Govt');

SELECT SchName, Amount FROM Scholarship
WHERE Amount > ANY (SELECT Amount FROM
Scholarship WHERE Provider = 'Central
Govt');

-- Sub-query in FROM (derived table)
SELECT Department, AvgCGPA
FROM (SELECT Department, AVG(CGPA) AS
AvgCGPA FROM Student GROUP BY Department)
AS d
WHERE AvgCGPA > 8;

```

```

-- CORRELATED : students whose CGPA is
above their own department's average
SELECT s.FirstName, s.Department, s.CGPA
FROM Student s
WHERE s.CGPA > (SELECT AVG(s2.CGPA) FROM
Student s2 WHERE s2.Department =
s.Department);

-- CORRELATED with EXISTS : students having
at least one approved application
SELECT s.FirstName
FROM Student s
WHERE EXISTS (SELECT 1 FROM Application a
              WHERE a.StudentID =
s.StudentID AND a.Status = 'Approved');

-- CORRELATED with NOT EXISTS : students
with no application at all
SELECT s.FirstName
FROM Student s
WHERE NOT EXISTS (SELECT 1 FROM Application
a WHERE a.StudentID = s.StudentID);

-- CORRELATED in SELECT list : number of
applications per student
SELECT s.FirstName,
       (SELECT COUNT(*) FROM Application a
WHERE a.StudentID = s.StudentID) AS
AppCount
FROM Student s;
GO

/*

```

```

=====
=====
2b a) VIEWS, SYNONYMS, SEQUENCES

=====
===== */

-- ----- VIEW (CREATE VIEW must be
first statement in its batch) -----
CREATE VIEW vw_ApprovedScholars AS
SELECT s.StudentID, s.FirstName,
s.LastName, sc.SchName, sc.Amount,
a.AppDate
FROM Student s
JOIN Application a ON s.StudentID =
a.StudentID
JOIN Scholarship sc ON a.ScholarshipID =
sc.ScholarshipID
WHERE a.Status = 'Approved';
GO

SELECT * FROM vw_ApprovedScholars;
GO

ALTER VIEW vw_ApprovedScholars AS      --
modify a view
SELECT s.StudentID, s.FirstName,
sc.SchName, sc.Amount
FROM Student s
JOIN Application a ON s.StudentID =
a.StudentID
JOIN Scholarship sc ON a.ScholarshipID =
sc.ScholarshipID
WHERE a.Status = 'Approved';

```

```

GO

-- View that can be updated (simple view,
WITH CHECK OPTION)
CREATE VIEW vw_CSE_Students AS
SELECT StudentID, FirstName, LastName,
Department, CGPA
FROM Student WHERE Department = 'CSE'
WITH CHECK OPTION;
GO

UPDATE vw_CSE_Students SET CGPA = 9.00
WHERE StudentID = 3;
DROP VIEW vw_CSE_Students;
GO

-- ----- SYNONYM -----
CREATE SYNONYM syn_Student FOR dbo.Student;
GO
SELECT FirstName, Department FROM
syn_Student;      -- works like the real
table
DROP SYNONYM syn_Student;
GO

-- ----- SEQUENCE -----
CREATE SEQUENCE seq_ReceiptNo
AS INT
START WITH 1000
INCREMENT BY 1
MINVALUE 1000
MAXVALUE 9999
NO CYCLE;
GO

```

```

SELECT NEXT VALUE FOR seq_ReceiptNo AS
Receipt1;
SELECT NEXT VALUE FOR seq_ReceiptNo AS
Receipt2;

-- Use sequence to generate ids
CREATE TABLE Receipt (
    ReceiptNo INT DEFAULT (NEXT VALUE FOR
seq_ReceiptNo) PRIMARY KEY,
    AppID      INT,
    IssuedOn   DATE DEFAULT GETDATE()
);
INSERT INTO Receipt (AppID) VALUES (1),
(2), (4);
SELECT * FROM Receipt;

ALTER SEQUENCE seq_ReceiptNo RESTART WITH
2000;
DROP TABLE Receipt;
DROP SEQUENCE seq_ReceiptNo;
GO

/*
=====
=====
2b  b) T-SQL BLOCKS : VARIABLES, TYPE
DECLARATIONS, LOOPS
=====
===== */

-- 1: Print odd numbers from 1 to N
DECLARE @N INT = 15, @i INT = 1;

```

```

PRINT 'Odd numbers from 1 to ' + CAST(@N AS
VARCHAR(5)) + ':';
WHILE @i <= @N
BEGIN
    IF @i % 2 <> 0
        PRINT @i;
    SET @i = @i + 1;
END
GO

-- 2: Fibonacci series up to a given number
DECLARE @Limit INT = 100,
        @a INT = 0,
        @b INT = 1,
        @next INT,
        @series VARCHAR(500) = '';
SET @series = CAST(@a AS VARCHAR(10)) + ' '
+ CAST(@b AS VARCHAR(10));
SET @next = @a + @b;
WHILE @next <= @Limit
BEGIN
    SET @series = @series + ' ' +
CAST(@next AS VARCHAR(10));
    SET @a = @b;
    SET @b = @next;
    SET @next = @a + @b;
END
PRINT 'Fibonacci up to ' + CAST(@Limit AS
VARCHAR(10)) + ': ' + @series;
GO

-- 3: Sum of first N natural numbers
DECLARE @n3 INT = 10, @k INT = 1, @sum INT
= 0;

```

```

WHILE @k <= @n3
BEGIN
    SET @sum += @k;
    SET @k += 1;
END
PRINT 'Sum of first ' + CAST(@n3 AS
VARCHAR(5)) + ' numbers = ' + CAST(@sum AS
VARCHAR(10));
PRINT 'Check by formula  $n(n+1)/2 =$  ' +
CAST(@n3 * (@n3 + 1) / 2 AS VARCHAR(10));
GO

-- 4: Reverse a given number
DECLARE @num INT = 12345, @rev INT = 0,
@digit INT, @orig INT;
SET @orig = @num;
WHILE @num > 0
BEGIN
    SET @digit = @num % 10;
    SET @rev = @rev * 10 + @digit;
    SET @num = @num / 10;
END
PRINT 'Reverse of ' + CAST(@orig AS
VARCHAR(10)) + ' is ' + CAST(@rev AS
VARCHAR(10));
GO

-- Bonus: variables with the scholarship
data (typed variables + SELECT INTO
variable)
DECLARE @name VARCHAR(30), @cgpa
DECIMAL(4,2), @dept VARCHAR(20);
SELECT @name = FirstName, @cgpa = CGPA,
@dept = Department

```

```

FROM Student WHERE StudentID = 6;
PRINT @name + ' (' + @dept + ') has CGPA '
+ CAST(@cgpa AS VARCHAR(6));

IF @cgpa >= 9
    PRINT 'Eligible for Merit Excellence
Award';
ELSE
    PRINT 'Not eligible for Merit
Excellence Award';
GO

/*
=====
=====

2c c) ERROR HANDLING AND EXCEPTION
HANDLING

=====
===== */

-- Basic TRY...CATCH with error functions
BEGIN TRY
    DECLARE @x INT = 10 / 0;
END TRY
BEGIN CATCH
    SELECT ERROR_NUMBER()    AS ErrNo,
           ERROR_SEVERITY()  AS Severity,
           ERROR_STATE()     AS ErrState,
           ERROR_PROCEDURE() AS ProcName,
           ERROR_LINE()      AS LineNo,
           ERROR_MESSAGE()   AS Msg;
END CATCH;
GO

```

```

-- Constraint violation handled gracefully
BEGIN TRY
    INSERT INTO Application
    (AppID,StudentID,ScholarshipID,Status)
    VALUES (9, 1, 101, 'Approved');
    INSERT INTO Application
    (AppID,StudentID,ScholarshipID,Status)
    VALUES (9, 1, 101,
    'Approved');          -- duplicate PK
END TRY
BEGIN CATCH
    PRINT 'Insert failed: ' +
    ERROR_MESSAGE();
END CATCH;
GO

-- Transaction + TRY/CATCH + rollback (all-
or-nothing disbursement)
BEGIN TRY
    BEGIN TRANSACTION;
        INSERT INTO Disbursement
    (AppID,Amount,DisbDate,Mode)
        VALUES (3, 40000, GETDATE(), 'Bank
    Transfer');
        UPDATE Application SET Status =
    'Approved' WHERE AppID = 3;
        INSERT INTO Disbursement
    (AppID,Amount,DisbDate,Mode)
        VALUES (3, -500, GETDATE(), 'Bank
    Transfer');          -- violates CHECK (Amount
    > 0)
        COMMIT TRANSACTION;
END TRY

```

```

BEGIN CATCH
    IF @@TRANCOUNT > 0 ROLLBACK
TRANSACTION;
    PRINT 'Transaction rolled back: ' +
ERROR_MESSAGE();
END CATCH;

SELECT AppID, Status FROM Application WHERE
AppID = 3;      -- still Pending
GO

-- Custom errors with RAISERROR and THROW
inside a stored procedure
CREATE PROCEDURE usp_ApplyScholarship
    @AppID INT, @StudentID INT, @SchID INT
AS
BEGIN
    SET NOCOUNT ON;
    DECLARE @cgpa DECIMAL(4,2), @min
DECIMAL(4,2);
    BEGIN TRY
        SELECT @cgpa = CGPA FROM Student
WHERE StudentID = @StudentID;
        SELECT @min = MinCGPA FROM
Scholarship WHERE ScholarshipID = @SchID;

        IF @cgpa IS NULL
            THROW 50001, 'Student does not
exist.', 1;
        IF @cgpa < @min
            RAISERROR('CGPA %s is below the
required minimum.', 16, 1, @cgpa);

        INSERT INTO Application

```

```

(AppID,StudentID,ScholarshipID) VALUES
(@AppID,@StudentID,@SchID);
    PRINT 'Application submitted
successfully.';
    END TRY
    BEGIN CATCH
        PRINT 'Error ' +
CAST(ERROR_NUMBER() AS VARCHAR(10)) + ': '
+ ERROR_MESSAGE();
    END CATCH
END;
GO

EXEC usp_ApplyScholarship 20, 99, 101;  --
student missing
EXEC usp_ApplyScholarship 21, 7, 101;   --
CGPA too low
EXEC usp_ApplyScholarship 22, 6, 104;   --
success
GO

-- Nested TRY...CATCH with XACT_STATE and
rethrow
BEGIN TRY
    BEGIN TRY
        SELECT CAST('abc' AS INT);      --
conversion error
    END TRY
    BEGIN CATCH
        PRINT 'Inner catch: ' +
ERROR_MESSAGE();
        THROW;                          --
rethrow to outer block
    END CATCH

```

```

END TRY
BEGIN CATCH
    PRINT 'Outer catch: ' +
ERROR_MESSAGE();
    PRINT 'XACT_STATE = ' +
CAST(XACT_STATE() AS VARCHAR(2));
END CATCH;
GO

/*
=====
=====
5. JSON DATATYPE (JSON stored as
NVARCHAR(MAX) with ISJSON check;
SQL Server 2025 also offers a native
JSON type)

=====
===== */

CREATE TABLE StudentProfile (
    StudentID INT PRIMARY KEY,
    ProfileJson NVARCHAR(MAX) CHECK
(ISJSON(ProfileJson) = 1),
    CONSTRAINT FK_Profile_Student FOREIGN
KEY (StudentID) REFERENCES
Student(StudentID)
);

INSERT INTO StudentProfile VALUES
(1, N'{"hobbies":
["chess","coding"],"address":
{"city":"Bengaluru","pin":"560001"},"achiev
ements":[{"title":"Hackathon

```

```

Winner", "year": 2025}, {"title": "Dean's
List", "year": 2026}]]') ,
(2, N'{"hobbies":["dance"],"address":
{"city":"Mysuru","pin":"570001"},"achievements":[]}') ,
(6, N'{"hobbies":
["reading","cricket"],"address":
{"city":"Hyderabad","pin":"500001"},"achievements":[{"title":"State
Topper", "year": 2026}]}') ;
GO

```

```

-- Validate
SELECT ISJSON('{ "a":1 }') AS Valid,
ISJSON('not json') AS Invalid;

```

```

-- JSON_VALUE : extract a scalar
SELECT StudentID,
       JSON_VALUE(ProfileJson,
'$ .address.city') AS City,
       JSON_VALUE(ProfileJson,
'$ .hobbies[0]') AS FirstHobby
FROM StudentProfile;

```

```

-- JSON_QUERY : extract an object / array
SELECT StudentID, JSON_QUERY(ProfileJson,
'$ .hobbies') AS HobbiesArray
FROM StudentProfile;

```

```

-- Filter on JSON property
SELECT StudentID FROM StudentProfile
WHERE JSON_VALUE(ProfileJson,
'$ .address.city') = 'Bengaluru';

```

```

-- JSON_MODIFY : update / add / remove
UPDATE StudentProfile
SET ProfileJson = JSON_MODIFY(ProfileJson,
'$address.city', 'Mangaluru')
WHERE StudentID = 2;

UPDATE StudentProfile
SET ProfileJson = JSON_MODIFY(ProfileJson,
'$phone', '9876543210')
WHERE StudentID = 1;

UPDATE StudentProfile
SET ProfileJson = JSON_MODIFY(ProfileJson,
'$phone', NULL)      -- removes key
WHERE StudentID = 1;

-- OPENJSON : shred JSON into rows
SELECT p.StudentID, h.[value] AS Hobby
FROM StudentProfile p
CROSS APPLY OPENJSON(p.ProfileJson,
'$hobbies') AS h;

SELECT p.StudentID, a.title, a.year
FROM StudentProfile p
CROSS APPLY OPENJSON(p.ProfileJson,
'$achievements')
    WITH (title NVARCHAR(50) '$.title',
    year INT '$.year') AS a;

-- Parse a JSON string directly
DECLARE @json NVARCHAR(MAX) =
N'[{ "id":201,"name":"Ravi","cgpa":8.4},
{ "id":202,"name":"Latha","cgpa":9.2}]';
SELECT * FROM OPENJSON(@json) WITH (id INT,

```

```

name VARCHAR(30), cgpa DECIMAL(4,2));

-- FOR JSON : relational rows -> JSON
SELECT StudentID, FirstName, Department,
CGPA
FROM Student WHERE CGPA > 8.5
FOR JSON PATH;

SELECT s.FirstName AS [student.name],
sc.SchName AS [student.scholarship],
a.Status AS [student.status]
FROM Student s
JOIN Application a ON s.StudentID =
a.StudentID
JOIN Scholarship sc ON a.ScholarshipID =
sc.ScholarshipID
FOR JSON PATH, ROOT('applications');

-- Violation test: invalid JSON is rejected
by the CHECK constraint
BEGIN TRY
    INSERT INTO StudentProfile VALUES (3,
N'this is not json');
END TRY
BEGIN CATCH PRINT 'JSON CHECK error: ' +
ERROR_MESSAGE(); END CATCH;
GO

```